Ders 7 Lab Çalışma Özeti

Ders 7 Lab Çalışma Özeti

Genel Konular

- Aggregate (Gruplama) Fonksiyonları
  - AVG: Ortalama değer hesaplama
  - COUNT: Kayıt/satır sayısını bulma (COUNT(*) tüm satırları sayar)
  - MIN / MAX: Sütundaki en küçük/en büyük değeri bulma
  - SUM: Belirli bir sütundaki değerlerin toplamını hesaplama
  - Bu fonksiyonlar sadece SELECT ve HAVING ifadelerinde kullanılabilir
  - WHERE koşulunda doğrudan kullanılamaz, iç içe sorgu (subquery) gerektirir
- GROUP BY (Gruplama)
  - Kayıtları belirli bir sütuna göre gruplar
  - Her grup üzerinde aggregate fonksiyon uygulanabilir
  - Çoklu gruplama yapılabilir (örn: departmana göre, ardından cinsiyete göre)
  - Arka planda: grup oluşturup her gruptaki kayıt sayısını/m değerlerini hesaplar
- ORDER BY (Sıralama)
  - Sorgu sonucunu belirli sütuna göre sıralar
  - ASC: Artan sıralama (küçükten büyüğe) — varsayılan
  - DESC: Azalan sıralama (büyükten küçüğe)
  - Çoklu sıralama yapılabilir (örn: önce ID'ye göre azalan, sonra koda göre artan)
- HAVING
  - Aggregate fonksiyonlarla koşul belirleme imkanı sunar
  - WHERE kullanılamayan yerlerde (aggregate fonksiyonlu koşullar) HAVING kullanılır
  - Mutlaka GROUP BY ile birlikte kullanılır
  - Örnek: "ortalama maaşı 40.000'den fazla olan departmanlar"
- LIMIT ve OFFSET
  - LIMIT: Sorgu sonucundan belirli sayıda satır döndürme
  - OFFSET: İlk N kaydı atlayarak sonuca başlama
  - Örnek: LIMIT 3 OFFSET 5 → 6. kayıttan itibaren 3 satır getir
  - En yüksek/değerli kaydı bulmak için ORDER BY + LIMIT 1 birlikte kullanılır
- IS NULL / IS NOT NULL
  - Bir sütundaki boş (NULL) değerleri filtreleme
  - Örnek: Yöneticisi olmayan kişileri bulma (Super_SSN IS NULL)
- EXTRACT Fonksiyonu
  - Date tipindeki verilerden spesifik bileşen çekme
  - Kullanılabilir bileşenler: YEAR, MONTH, DAY, CENTURY, WEEK
  - Örnek: EXTRACT(YEAR FROM B_date)

Hocanın Özellikle Vurguladığı Kısımlar

- Aggregate fonksiyonlar WHERE□□ doğrudan kullanılamaz
  - WHERE koşulunda MAX(B_date) gibi bir ifade yazılamaz
  - Çözüm: İç içe sorgu (subquery) kullanmak gerekir veya HAVING ifadesi tercih edilir
- HAVING tek başına kullanılamaz
  - Mutlaka önce GROUP BY olmalıdır
  - Mantık: Önce grupla, sonra gruplar üzerinde koşul uygula
- GROUP BY'nin arka plan mantığı
  - Gruplama yapılırken arka planda geçici bir tablo oluşur

  - Her grup için ayrı satırlar oluşturulur ve aggregate fonksiyonlar bu gruplar üzerinde çalışır
- Sütun isimlendirmesi
  - AS keyword'ü ile sütunlara takma isim verilebilir (örn: AVG(salary) AS ortalama_maas)
  - AS kullanmadan da doğrudan fonksiyon yanına isim yazılabilir
  - Tablolarda olduğu gibi sütunlarda da takma isim verilebilir
- ORDER BY ve GROUP BY birbirinden bağımsızdır
  - Herhangi bir sorgu sonucu sıralanabilir, GROUP BY olmasa bile
- LIMIT 1 kullanımı
  - OFFSET sıfır olarak kabul edilir
  - En yüksek/değerli tek bir kaydı bulmak için ideal yoldur

Kısa Tekrar Notları

- Aggregate fonksiyonlar: AVG, COUNT, MIN, MAX, SUM
- GROUP BY: Kayıtları grupla, her grup için hesaplama yap
- ORDER BY: ASC (artan, varsayılan) / DESC (azalan)
- HAVING: Aggregate koşulları için WHERE alternatifi, GROUP BY ile birlikte kullanılır
- Sorgu sırası: SELECT → FROM → WHERE → GROUP BY → HAVING → ORDER BY → LIMIT
- LIMIT a OFFSET b: b+1. kayıttan itibaren a satır getir
- IS NULL: Boş değerleri filtrele
- EXTRACT(YEAR FROM date_sütunu): Tarihten yıl bilgisini çek
- WHERE'de aggregate kullanılamaz → iç içe sorgu veya HAVING kullan

Detaylı Açıklamalar

- **Aggregate Fonksiyonların Kullanım Alanları:** Bu fonksiyonlar, veritabanındaki büyük veri kümelerinden anlamlı istatistiksel bilgiler çıkarmak için kullanılır. Örneğin, bir departmanın ortalama maaşını bulmak için AVG(salary), bir projede kaç kişi çalıştığını öğrenmek için COUNT(*) kullanılır. Bu fonksiyonlar sadece SELECT listesinde ve HAVING koşulunda kullanılabilir.
- **GROUP BY ile Çoklu Gruplama:** Birden fazla sütuna göre gruplama yapılabilir. Örneğin, departmana göre ve ardından cinsiyete göre gruplama yaparak "hangi departmanda kaç erkek/kadın çalışan var" sorusuna cevap bulunabilir. Sorgu sonucu her grup için ayrı satır olarak döner.
- **WHERE vs. Fark:** WHERE, aggregate fonksiyonlar kullanmadan önce satırları filtreler. HAVING ise gruplandırma yapıldıktan sonra gruplar üzerinde filtreleme yapar. WHERE'de aggregate kullanılamaz ama HAVING'de kullanılabilir. Örneğin: "ortalama maaşı 40.000'den fazla olan departmanları bul" için HAVING kullanılır.
- **İç İçe Sorgu ile WHERE'de Aggregate Kullanımı:** WHERE koşulunda aggregate fonksiyon kullanılmak istenirse, iç içe sorgu (subquery) yazılır. Örnek: `WHERE B_date = (SELECT MAX(B_date) FROM Employee)` şeklinde bir yazım gereklidir. Doğrudan `WHERE B_date = MAX(B_date)` yazımı hata verir.
- **ORDER BY ile Çoklu Sıralama:** Birden fazla sütuna göre sıralama yapılabilir. İlk belirtilen sütun öncelikli olarak sıralanır, aynı değere sahip kayıtlar ise ikinci sütuna göre sıralanır. Örnek: `ORDER BY ID DESC, Kod ASC` → önce ID'ye göre büyükten küçüğe, ardından koda göre küçükten büyüğe sıralama.
- **LIMIT ve OFFSET ile Sayfalama:** Büyük veri kümelerinden belirli aralıklarda veri çekmek için kullanılır. `LIMIT 3 OFFSET 5` ifadesi, ilk 5 kaydı atlayarak 6., 7. ve 8. kayıtları getirir. Bu özellik özellikle sayfalama (pagination) uygulamalarında önemlidir.
- **IS NULL ile Eksik Veri Analizi:** Veritabanında boş kalan alanları tespit etmek için kullanılır. Örneğin, yöneticisi olmayan (Super_SSN değeri NULL olan) çalışanları bulmak için `WHERE Super_SSN IS NULL` koşulu kullanılır. Tersi olarak IS NOT NULL ile dolu olan kayıtlar filtrelenir.

- **EXTRACT ile Tarih İşlemleri:** Date tipindeki sütunlardan yıl, ay, gün gibi bileşenleri çekmek için EXTRACT fonksiyonu kullanılır. Bu fonksiyon, tarih bazlı analizlerde ve raporlamalarda işe yarar. Örnek: `EXTRACT(YEAR FROM B_date)` ifadesi, doğum tarihinden yılı çeker.
- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.